MODULE *sync_scheduler*

Version 2 block-download scheduler specification.

Models the layer above the per-connection protocol in *protocol.tla*: the node choosing which peer to ask for each block — the layer where Zebra's genesis-to-tip sync stall lived in the legacy protocol (*ZcashFoundation*/zebra ≠ 10679; see .. /*sync_scheduler.tla*). Zebra's *v*2 stack defers this layer to a future scheduler ("ibd-engine"), so this module models it ahead of the implementation.

The *v*2 protocol gives a request more ways to end without a block than the legacy protocol did. Only one of them says anything about the peer's chain:

- not-found — the per-entry result 0*x*02 of get-blocks: an explicit statement that the peer lacks the block.
- REFUSED — the responder reset the stream, declining to serve it (overload, rate limits, Zebra's bulk-stream cap). Says nothing about the peer's chain.
- truncation — the responder finished after any complete entry, leaving later entries unanswered ("get-blocks", "get-block-range"). Routine and sanctioned; says nothing about the peer's chain.
- timeout — no response within the stall timeout; the requester cancels with *CANCELLED* ("Block Download Parameters"). Says nothing about the peer's chain.

Three booleans mark each of the uninformative outcomes as informative, each reproducing a variant of the legacy registry-poisoning bug:

- *TreatRefusedAsMissing* = TRUE → a REFUSED marks the peer missing
- *TreatTruncatedAsMissing* = TRUE → an unanswered entry marks the peer missing
- *BuggyTimeout* = TRUE → a timeout marks the peer missing (the exact legacy zebra ≠ 10679 bug)

Separately, the scheduler applies Zebra's unresponsive-peer rule: a peer that times out *UnresponsiveLimit* requests in a row, with no response of any kind in between, is disconnected (*ZcashFoundation*/zebra ≠ 11276). The *Redial* boolean decides whether disconnected peers are ever redialled; without it, evicting the only holder of a block stalls the sync even though every registry entry is honest.

RegistryHonest and *EventuallyAllVerified* hold with every switch in its fixed position (*Redial* = TRUE, the rest FALSE) and are violated under the buggy settings, each as a short *TLC* counterexample.

See .. /documents/v2-*modeling.md* for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *Peers*	set of peer ids
CONSTANT *MaxBlock*	blocks to download are 1 .. *MaxBlock*
CONSTANT *LaggingPeers*	peers that hold only blocks 1 .. *LagTip*
CONSTANT *LagTip*	chain height of a lagging peer (< *MaxBlock*)
CONSTANT *MaxRetries*	bound on re-queue attempts per block after a notfound
CONSTANT *UnresponsiveLimit*	consecutive timeouts before a peer is disconnected
CONSTANT *TreatRefusedAsMissing*	TRUE treats a REFUSED as a notfound (buggy)
CONSTANT *TreatTruncatedAsMissing*	TRUE treats an unanswered entry as a notfound (buggy)
CONSTANT *BuggyTimeout*	TRUE treats a timeout as a notfound (legacy bug)
CONSTANT *Redial*	TRUE redials disconnected peers

$Blocks \triangleq 1 \,..\, MaxBlock$

$NONE \triangleq$ "none" sentinel for "no peer"; must not be a member of *Peers*

Ground truth: a lagging peer holds blocks up to *LagTip*; every other peer is at tip and holds them all. The scheduler never reads this directly — it learns a peer's contents only through the outcomes below.

$PeerTip(p) \triangleq$ IF $p \in LaggingPeers$ THEN $LagTip$ ELSE $MaxBlock$

$Holds(p,\ b) \triangleq b \leq PeerTip(p)$

VARIABLES

verified,	subset of *Blocks* downloaded and verified (the frontier)
pending,	subset of *Blocks* the scheduler still wants and may request
inflight,	[$Blocks \to Peers \cup \{NONE\}$] peer a block is requested from
avail,	[$Peers \to$ [$Blocks \to$ {"has", "missing", "unknown"}]] registry
retries,	[$Blocks \to 0\,..\,MaxRetries$] re-queue attempts used
timeouts,	[$Peers \to 0\,..\,UnresponsiveLimit$] consecutive unanswered timeouts
connected	[$Peers \to$ BOOLEAN] FALSE once disconnected as unresponsive

$vars \triangleq \langle verified,\ pending,\ inflight,\ avail,\ retries,\ timeouts,\ connected \rangle$

$Init \triangleq$

$\land\ verified = \{\}$
$\land\ pending = Blocks$
$\land\ inflight = [b \in Blocks \mapsto NONE]$
$\land\ avail = [p \in Peers \mapsto [b \in Blocks \mapsto \text{"unknown"}]]$
$\land\ retries = [b \in Blocks \mapsto 0]$
$\land\ timeouts = [p \in Peers \mapsto 0]$
$\land\ connected = [p \in Peers \mapsto \text{TRUE}]$

The scheduler opens a get-blocks request for a pending block toward a connected peer not already marked missing for it.

$Request \triangleq$

$\exists\, b \in pending :$
  $\exists\, p \in Peers :$
    $\land\ inflight[b] = NONE$
    $\land\ connected[p]$
    $\land\ avail[p][b] \neq \text{"missing"}$
    $\land\ inflight' = [inflight \text{ EXCEPT } ![b] = p]$
    $\land\ pending' = pending \setminus \{b\}$
    $\land$ UNCHANGED $\langle verified,\ avail,\ retries,\ timeouts,\ connected \rangle$

The in-flight peer genuinely has the block: it is delivered and verified. Any response resets the peer's consecutive-timeout count.

$DeliverBlock \triangleq$

$\exists\, b \in Blocks :$
  LET $p \triangleq inflight[b]$ IN

$\land\ p \neq NONE$
$\land\ Holds(p,\ b)$
$\land\ verified' = verified \cup \{b\}$
$\land\ avail' = [avail \text{ EXCEPT } ![p][b] = \text{“has”}]$
$\land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\land\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\land$ UNCHANGED $\langle pending,\ retries,\ connected\rangle$

The in-flight peer genuinely lacks the block and answers the entry with the explicit not-found result. This is the one outcome that legitimately marks the registry, and the block is re-queued (bounded) toward other peers.

$NotFound \triangleq$
$\exists\, b \in Blocks :$
LET $p \triangleq inflight[b]$ IN
$\land\ p \neq NONE$
$\land\ \neg Holds(p,\ b)$
$\land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\land\ avail' = [avail \text{ EXCEPT } ![p][b] = \text{“missing”}]$
$\land\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\land$ IF $retries[b] < MaxRetries$
THEN $\land\ pending' = pending \cup \{b\}$
$\land\ retries' = [retries \text{ EXCEPT } ![b] = @ + 1]$
ELSE $\land$ UNCHANGED $pending$ retries exhausted
$\land$ UNCHANGED $retries$
$\land$ UNCHANGED $\langle verified,\ connected\rangle$

The responder resets the request stream with REFUSED — it declines to serve the request, whether or not it holds the block (draft: “Request Streams”; Zebra refuses beyond 2 concurrent bulk streams). A REFUSED is a response, so the timeout count resets; what it does to the registry is the *TreatRefusedAsMissing* switch.

$Refused \triangleq$
$\exists\, b \in Blocks :$
LET $p \triangleq inflight[b]$ IN
$\land\ p \neq NONE$
$\land\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\land\ pending' = pending \cup \{b\}$
$\land\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\land\ avail' = [avail \text{ EXCEPT } ![p][b] =$
IF $TreatRefusedAsMissing$ THEN “missing” ELSE $@]$
$\land$ UNCHANGED $\langle verified,\ retries,\ connected\rangle$

The responder finishes the stream early, leaving this entry unanswered (draft: “get-blocks” / “get-block-range” allow finishing after any complete entry; truncation is resumable). Sanctioned and routine — but uninformative about the peer’s chain. The registry effect is the *TreatTruncatedAsMissing*

switch.

$Truncated \triangleq$
$\exists\, b \in Blocks :$
LET $p \triangleq inflight[b]$ IN
$\wedge\ p \neq NONE$
$\wedge\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\wedge\ pending' = pending \cup \{b\}$
$\wedge\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\wedge\ avail' = [avail \text{ EXCEPT } ![p][b] =$
IF $TreatTruncatedAsMissing$ THEN "missing" ELSE @]
$\wedge$ UNCHANGED $\langle verified, retries, connected\rangle$

No response within the stall timeout: the requester cancels the stream with *CANCELLED* and re-queues the block (draft: "Block Download Parameters"). The registry effect is the *BuggyTimeout* switch — the exact legacy bug. The peer's consecutive-timeout count rises; at *UnresponsiveLimit* the peer is disconnected as unresponsive (Zebra's rule).

$Timeout \triangleq$
$\exists\, b \in Blocks :$
LET $p \triangleq inflight[b]$ IN
$\wedge\ p \neq NONE$
$\wedge\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\wedge\ pending' = pending \cup \{b\}$
$\wedge\ avail' = [avail \text{ EXCEPT } ![p][b] =$
IF $BuggyTimeout$ THEN "missing" ELSE @]
$\wedge\ timeouts' = [timeouts \text{ EXCEPT } ![p] = \text{IF } @ < UnresponsiveLimit \text{ THEN } @ + 1 \text{ ELSE } @]$
$\wedge\ connected' = [connected \text{ EXCEPT } ![p] = @ \wedge timeouts[p] + 1 < UnresponsiveLimit]$
$\wedge$ UNCHANGED $\langle verified, retries\rangle$

A disconnected peer is redialled (Zebra maintains connections to its initial and discovered peers, redialling them when they fail). Gated by the *Redial* switch; without it eviction is forever.

$Reconnect \triangleq$
$\exists\, p \in Peers :$
$\wedge\ Redial$
$\wedge\ \neg connected[p]$
$\wedge\ connected' = [connected \text{ EXCEPT } ![p] = \text{TRUE}]$
$\wedge\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\wedge$ UNCHANGED $\langle verified, pending, inflight, avail, retries\rangle$

$Next \triangleq$
$\vee\ Request$
$\vee\ DeliverBlock$
$\vee\ NotFound$

$\lor$ *Refused*
$\lor$ *Truncated*
$\lor$ *Timeout*
$\lor$ *Reconnect*

Strong fairness on the progress actions encodes the real-world assumption that the node keeps trying, a willing peer eventually responds, and the redial loop keeps running. Refused, *Truncated* and *Timeout* get no fairness — they are adversarial but never forced.

$Spec \triangleq$
$Init$
$\land \Box[Next]_{vars}$
$\land \mathrm{SF}_{vars}(Request)$
$\land \mathrm{SF}_{vars}(DeliverBlock)$
$\land \mathrm{SF}_{vars}(NotFound)$
$\land \mathrm{SF}_{vars}(Reconnect)$

Liveness: every block is eventually downloaded and verified.

$EventuallyAllVerified \triangleq \Diamond(verified = Blocks)$

Safety invariants.

$TypeOK \triangleq$
$\land \quad verified \subseteq Blocks$
$\land \quad pending \subseteq Blocks$
$\land \quad inflight \in [Blocks \to Peers \cup \{NONE\}]$
$\land \quad avail \in [Peers \to [Blocks \to \{\text{“has”}, \text{“missing”}, \text{“unknown”}\}]]$
$\land \quad retries \in [Blocks \to 0 \,..\, MaxRetries]$
$\land \quad timeouts \in [Peers \to 0 \,..\, UnresponsiveLimit]$
$\land \quad connected \in [Peers \to \text{BOOLEAN }]$

The registry never marks a peer missing for a block it actually holds: only the explicit not-found result may mark it. “A refusal, a truncated response, and a timeout are not notfounds.”

$RegistryHonest \triangleq$
$\forall\, p \in Peers :$
$\forall\, b \in Blocks :$
$avail[p][b] = \text{“missing”} \Rightarrow \neg Holds(p,\ b)$

A peer is only ever recorded as having a block it genuinely holds.

$RegistryHasIsSound \triangleq$
$\forall\, p \in Peers :$
$\forall\, b \in Blocks :$
$avail[p][b] = \text{“has”} \Rightarrow Holds(p,\ b)$

A verified block was genuinely available from some peer.

$$
\begin{array}{l}
\mathit{VerifiedAreReal} \triangleq \\
\quad \forall\, b \in \mathit{verified} : \\
\qquad \exists\, p \in \mathit{Peers} : \mathit{Holds}(p,\, b)
\end{array}
$$